Python f-String

A formatted string, or f-string in Python, offers a modern and efficient way of string formatting. Introduced in Python 3.6, f-strings use curly braces ({ and }) to embed Python expressions inside string literals.

How to Format Strings in Python

The most modern and recommended way to format strings in Python is by using f-strings (formatted string literals). To create an f-string, prefix the string with the letter f and place variables or expressions inside curly braces {} directly in the string.

Syntax:

An f-string is created by adding f before the string and placing variables or expressions inside {}.

```
f"{variable/expression}"
```

Examples

The following example defines an f-string without variables or expressions inside.

```
1 txt = f"The price is 15 dollars"
2 print(txt)
```

Ex_0501.py

```
The price is 15 dollars
```

In the following example, variables are placed inside an f-string.

```
1 name = "Sara"
2 age = 20
3 greeting = f"Hello, {name}. You are {age} years old."
4 print(greeting)
```

Ex_0502.py

```
Hello, Sara. You are 20 years old.
```

In the following example an expression is evaluated directly inside an f-string.

```
1 Math = 89
2 Physics = 91
3 English_language = 85
4 print(f"Mary got total marks {Physics + Math + English_language} out of 300")
```

Ex_0503.py

```
Mary got total marks 265 out of 300
```

In the following example variables are placed inside an f-string. The floating-point variable total_cost is formatted with two decimal places using the :.2f format specifier.

Ex_0504.py

```
1 name = "Majed"
2 item_count = 5
3 price = 19.99
4
5 # Basic f-string usage
6 print(f"Hello, {name}.")
7
8 # Using expressions and number formatting
9 total_cost = item_count * price
10 print(f"Your total is ${total_cost:.2f} for {item_count} items.")
```

```
Hello, Majed.
Your total is $99.95 for 5 items.
```

Augmented Assignment Operators

The operators +, -, *, /, %, // and ** can be combined with the assignment operator to form augmented operators. The following Table shows the *augmented assignment operators*.

Operator	Example Usage	Equivalent To
+=	x += 5	x = x + 5
-=	y −= 2	y = y − 2
*=	z *= 10	z = z * 10
/=	a /= b	a = a / b
%=	c %= 3	c = c % 3
//=	x //= 3	x = x // 3
**=	y **= 2	y = y**2

Generating Random Numbers

To generate a random integer in Python, the most straightforward and common approach is using the built-in random.randint(a, b) function, which returns a random integer including both endpoints.

Example

Ex_0505.py

```
1 import random
2 random_number = random.randint(10, 99)
3 print(random_number)
```

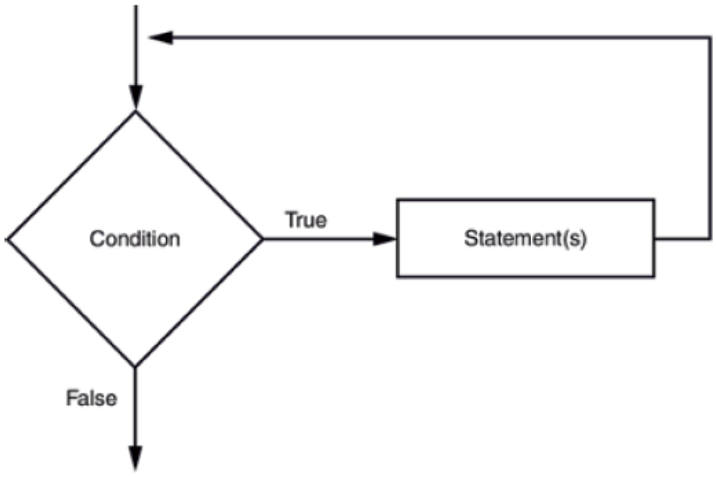
Repetition Structures (Loops)

Programmers commonly have to write code that performs the same task over and over. Instead of writing the same sequence of statements over and over, a better way to repeatedly perform an operation is to write the code for the operation once, then place that code in a structure that makes the computer repeat it as many times as necessary. This can be done with a repetition structure, which is more commonly known as a loop.

The concept of looping is fundamental to programming. Python provides two types of loop statements: while loops and for loops. The while loop is a *condition-controlled loop*; it is controlled by a true/false condition. The for loop is a *count-controlled loop* that repeats a specified number of times.

The while Loop

A while loop executes statements repeatedly as long as a condition remains true.



The general syntax of while statement is as follows.

```
while condition:
    Statement_1
    Statement_2
    Statement_3
……
```

The following examples demonstrate while loop

Write a Python program to print the first 10 natural numbers.

```python
# Print the first 10 natural numbers
num = 1
while num <= 10:
    print(num, end = ', ')
    num += 1
```

Ex_0506.py

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
```

Write a Python program to print the first 10 natural numbers in reverse order.

```python
# Print the first 10 natural numbers in reverse order
num = 10
while num >= 1:
    print(num, end = ', ')
    num -= 1
```

Ex_0507.py

```
10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
```

Write a Python program that prompts the user to enter 10 numbers, calculates their average, and displays the result.

```python
# Sheet (4), Q #01
# Calculate the average 10 input numbers
Sum = 0
i = 0
while i < 10:
    num = int(input("Enter number: "))
    Sum = Sum + num
    i = i + 1
print("Average = ", format(Sum/10,"<8.3f"))
```

Ex_0508.py

```
Enter number: 84
Enter number: 91
Enter number: 89
Enter number: 93
Enter number: 79
Enter number: 86
Enter number: 77
Enter number: 83
Enter number: 69
Enter number: 75
Average =  82.600
```

Write a Python program to print all even numbers falls between two numbers entered by the user.

```python
# Print all even numbers that falls between two numbers entered by the user
# exclusive both numbers
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
if num1 > num2:
    while num1 >= num2:
        if num2 % 2 == 0:
            print(num2)
        num2 += 1
else:
    while num1 <= num2:
        if num1 % 2 == 0:
            print(num1)
        num1 += 1
```

Ex_0509.py

The break Statement

With the break statement we can stop the loop even if the while condition is true.

```python
n = 0
while True:
    if 2 ** (n + 1) >= 30000:
        break
    n += 1

# Display the result
print(f"The largest integer n is: {n}")
print(f"Verification: 2^{n} = {2 ** n} (which is less than 30,000)")
print(f"Next power: 2^{n + 1} = {2 ** (n + 1)} (which exceeds 30,000)")
```

Ex_0510.py

```
The largest integer n is: 14
Verification: 2^14 = 16384 (which is less than 30,000)
Next power: 2^15 = 32768 (which exceeds 30,000)
```

Create a Python program that randomly selects a number between 1 and 10. Asks the user to guess the number. Tells the user if they guessed too high, too low, or correctly. Ends the game when they guess correctly.

Ex_0511.py

```
import random
n = 0
target = random.randint(1, 20)
while True:
    n +=1
    guess = int(input("Guess a number between 1 and 10: "))

    if guess < target:
        print("Too low! Try again.")
    elif guess > target:
        print("Too high! Try again.")
    else:
        print(f"Congratulations! You guessed it right after {n} Trials.")
        break
```

range Function

The Python range() function is a built-in tool used to generate a sequence of integers. It is most commonly used to control the number of iterations in a for loop.

The general syntax of range() function is as follows

```
range(start, stop, step)
```

start: it is an optional integer value representing the starting value of the sequence. The default value is 0.

stop: it is a required integer value representing the upper limit of the range.

step: it is an optional integer value representing the increment/decrement value. The default value is 1.

The for Loop

You use the for statement to write a count-controlled loop. In Python, the for statement is designed to work with a sequence of data items. When the statement executes, it iterates once for each item in the sequence. Here is the general format:

The general syntax of for statement is as follows.

```
for variable in range(start, stop, step):
        Statement_1
        Statement_2
        Statement_3
……
```

Examples

Write for loop statement to print the following series: 10. 20. 30 . …. 300

```
1 for i in range(10, 101, 10):
2     print(i, end =', ')
```

Ex_0512.py

```
10, 20, 30, 40, 50, 60, 70, 80, 90, 100,
```

Write for loop statement to print the following series: 105. 98. 91 . …. 7

```
1 for i in range(105, 7-1, -7):
2     print(i, end=', ')
```

Ex_0513.py

```
105, 98, 91, 84, 77, 70, 63, 56, 49, 42, 35, 28, 21, 14, 7,
```

Write a Python program that prompts the user to enter a non-negative integer, calculates its factorial, and displays the result.

```
1 fact = 1
2 num = int(input("Enter a number: "))
3 for i in range(1,num + 1):
4     fact = fact * i
5 print("The factorial of",num,"is",fact)
```

Ex_0514.py

```
Enter a number: 10
The factorial of 10 is 3628800
```

Write a Python program to find the sum of the following series: $1 + 8 + 27 + \ldots$ n-terms

Ex_0515.py

```python
# Take input from the user
n = int(input("Enter the number of terms (n): "))

# Initialize total_sum to 0
total_sum = 0

for i in range(1, n + 1):
    total_sum += i ** 3

print(f"The sum of the series up to {n} terms is: {total_sum}")
```

```
Enter the number of terms (n): 10
The sum of the series up to 10 terms is: 3025
```

Write a Python program to calculate and print the sum of the first n terms of the following series

$$\frac{1}{1!}+\frac{1}{2!}+\frac{1}{3!}+\ldots+\frac{1}{n!}$$

Ex_0516.py

```python
num = int(input("Enter any number: "))
Sum = 0
fact = 1

# The for loop replaces the while loop and automatically handles incrementing i
for i in range(1, num):
    fact = fact * i
    Sum = Sum + i / fact

print("Sum = ", round(Sum, 2))
```

```
Enter any number: 10
Sum =  2.72
```